

**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES**  
**SIMATS ENGINEERING**  
**COMPUTER SCIENCE AND ENGINEERING**  
**CSA17 - ARTIFICIAL INTELLIGENCE**

**Experiment No. 8: Python Program to Implement Depth-First Search (DFS)**

---

### **Aim**

To write a Python program to implement the Depth-First Search (DFS) algorithm for traversing a graph, exploring as far as possible along each branch before backtracking, using recursion.

### **Algorithm**

1. Start.
2. Represent the graph as an adjacency list using a dictionary, where each key is a node and its value is the list of adjacent (neighboring) nodes.
3. Read the starting node from which the traversal should begin.
4. Create an empty visited set and an empty list 'order' to store the sequence in which nodes are visited.
5. Define a recursive function dfs(graph, node, visited, order): a) Add the current node to the visited set and append it to 'order'. b) For each neighbor of the current node in the adjacency list, if it has not been visited, recursively call dfs on that neighbor.
6. Call dfs() with the starting node to begin the traversal.
7. The recursion continues to go deeper into unvisited neighbors until no further unvisited neighbor exists, at which point it backtracks to the previous node and tries its next unvisited neighbor.
8. Repeat until all nodes reachable from the starting node have been visited.
9. Print the final DFS traversal order.
10. Stop.

### **Pseudocode**

```
BEGIN
  INPUT: graph (adjacency list), start_node
  CREATE VISITED = {}
  CREATE order = []

  FUNCTION dfs(node)
    ADD node TO VISITED
    APPEND node to order

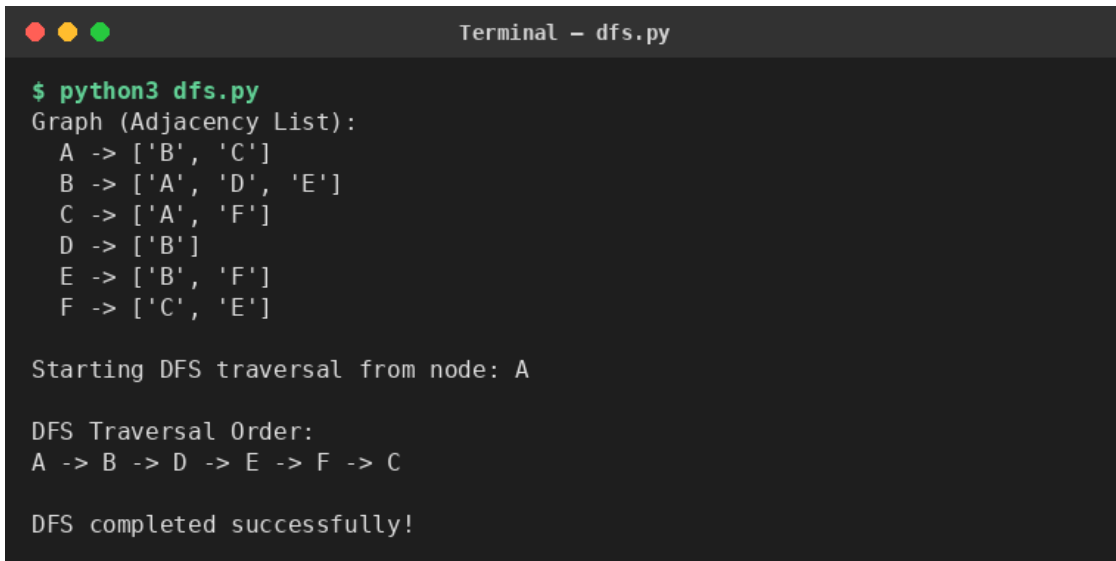
    FOR each neighbor IN graph[node] DO
      IF neighbor NOT IN VISITED THEN
        CALL dfs(neighbor)
      END IF
    END FOR
  END FUNCTION

  CALL dfs(start_node)
  PRINT order
END
```

## Result

The Python program to implement Depth-First Search was executed successfully. For the given graph with nodes A through F, starting the traversal from node A, the program recursively explored each branch as deep as possible before backtracking, producing the traversal order A -> B -> D -> E -> F -> C. Thus, the Depth-First Search algorithm was implemented and verified successfully using Python.

### Output Screenshot:

A terminal window titled "Terminal - dfs.py" with a dark background and light green text. It shows the execution of a Python script for Depth-First Search. The output displays the graph structure as an adjacency list, the starting node, the traversal order, and a success message.

```
Terminal - dfs.py

$ python3 dfs.py
Graph (Adjacency List):
A -> ['B', 'C']
B -> ['A', 'D', 'E']
C -> ['A', 'F']
D -> ['B']
E -> ['B', 'F']
F -> ['C', 'E']

Starting DFS traversal from node: A

DFS Traversal Order:
A -> B -> D -> E -> F -> C

DFS completed successfully!
```